UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

FACULTAD DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN Y CIENCIAS DE LA COMPUTACIÓN

INFORME TÉCNICO Y ACADÉMICO

PROYECTO: SISTEMA INTEGRAL DE OPTIMIZACIÓN DE RUTAS MÍNIMAS EN LA RED VIAL DEPARTAMENTAL (RutaQ)

Modelado Matemático, Implementación Algorítmica (Dijkstra, Floyd-Warshall, Yen) y Validación Computacional sobre la Red Vial de Quetzaltenango

Curso: _____

Catedrático: _____

Estudiante: Cristopher Alexis Castellanos Paz
Carné: 2690245972
Carrera: Ingeniería en Sistemas de Información y Ciencias de la Computación
Fecha: 14 de octubre de 2026
Lugar: Quetzaltenango, Guatemala
Aplicación Desplegada: https://criss-paz.github.io/rutaq/
Repositorio de Código: https://github.com/Criss-paz/rutaq

Resumen Ejecutivo

El presente trabajo de investigación aplicada y desarrollo tecnológico aborda la formulación matemática, diseño arquitectónico, implementación de software y verificación rigurosa del problema de la ruta más corta (*Shortest Path Problem*) aplicado a la red vial del departamento de Quetzaltenango, Guatemala. Se desarrolló la plataforma web reactiva **RutaQ**, la cual integra dos enfoques fundamentales de optimización sobre grafos ponderados: el algoritmo de Dijkstra para rutas desde origen único (*Single-Source Shortest Path*, SSSP) y el algoritmo de Floyd-Warshall para rutas entre todos los pares de vértices (*All-Pairs Shortest Path*, APSP), complementados con el algoritmo de Yen para la enumeración de los K caminos más cortos sin lazos y técnicas de majorización de estrés métrico (SMACOF) para la proyección visual del grafo en dos dimensiones.

El modelo se alimenta de un dataset de 24 cabeceras municipales oficiales del departamento de Quetzaltenango, construido mediante metodologías 100 % guiadas por datos a partir de OpenStreetMap, Nominatim y el motor Open Source Routing Machine (OSRM). Dicho grafo consta de 55 aristas directas aceptadas tras evaluar exhaustivamente los 276 pares candidatos posibles y filtrar aquellos con intermediaciones urbanas a menos de 1.0 km. La solución algorítmica fue sometida a un proceso de verificación exhaustivo de tres niveles: (1) validación paso a paso de la trazabilidad tabular contra una resolución analítica manual sobre un grafo de control de 6 nodos, (2) verificación cruzada automatizada al 100 % (576/576 pares) contra la librería científica NetworkX, y (3) contraste de una muestra de 20 distancias con Google Maps (13 aristas dentro de la tolerancia de $\pm 10\,\%$ y 7 corregidas con el valor de Google Maps al superar dicho umbral por selección de travesías urbanas). Los resultados demuestran la exactitud absoluta de las implementaciones, evidenciando cómo la interacción armónica entre teoría de grafos, estructuras de datos eficientes (montículos binarios) y arquitecturas desacopladas mediante Web Workers permite resolver redes viales complejas, garantizando fluidez interactiva y trazabilidad pedagógica.

1. Introducción

El transporte terrestre y la planificación de redes viales constituyen uno de los pilares socioeconómicos fundamentales para el comercio, la movilidad ciudadana y la logística de distribución en cualquier región geográfica. En territorios con topografía montañosa y accidentada, como el altiplano occidental guatemalteco, la determinación de rutas óptimas no puede basarse en aproximaciones euclidianas o intuitivas en línea recta. Factores geográficos como la orografía de la Sierra Madre, la presencia de fallas tectónicas, cañones fluviales y carreteras sinuosas demandan modelos de optimización basados en teoría de grafos que reflejen con precisión métrica las distancias y tiempos de traslado reales sobre la infraestructura vial existente.

Desde el punto de vista de las ciencias de la computación y la investigación de operaciones, el problema de la ruta más corta representa uno de los problemas fundacionales de optimización combinatoria. Su resolución algorítmica abarca desde técnicas codiciosas (*greedy algorithms*) como el algoritmo propuesto por Edsger W. Dijkstra en 1959, hasta esquemas basados en programación dinámica de abajo hacia arriba (*bottom-up dynamic programming*) como los formulados independientemente por Robert W. Floyd y Stephen Warshall en 1962. A pesar de su madurez teórica, la traslación de estos algoritmos a sistemas de software interactivos modernos plantea desafíos no triviales: la elección adecuada de estructuras de datos para mantener cotas asintóticas óptimas, la necesidad de proporcionar una visualización pedagógica de las etapas intermedias de cómputo (animaciones paso a paso, matrices evolutivas de recurrencia), y la integración de arquitecturas concurrentes que eviten el bloqueo de la interfaz de usuario durante búsquedas exhaustivas combinatorias.

El proyecto **RutaQ** surge como una respuesta técnica y académica a estas exigencias. La plataforma se encuentra desplegada y disponible para acceso público en **https://criss-paz.github.io/rutaq/**, y su código fuente completo está versionado bajo licencia abierta en el repositorio **https://github.com/Criss-paz/rutaq**. Desarrollada como una plataforma interactiva web en TypeScript y React, la herramienta permite no solamente calcular de forma instantánea el camino de costo mínimo entre cualquier par de municipios del departamento de Quetzaltenango, sino también auditar minuciosamente el proceso analítico mediante cuatro modalidades de visualización:

1. **Modalidad Tabular (S1):** Reproduce con exactitud la tabla de iteraciones clásica enseñada en cursos universitarios de Investigación de Operaciones y Teoría de Grafos, detallando la fijación de etiquetas temporales y permanentes, el tachado de columnas y la relajación de aristas.
2. **Modalidad Gráfica (S2):** Destaca de forma animada el árbol de caminos mínimos (*Shortest Path Tree*, SPT) o la ruta directa sobre el lienzo interactivo del grafo departamental.
3. **Modalidad de Rutas Alternativas (S3):** Enumera caminos simples ordenados por distancia empleando exploración en profundidad (DFS) y el algoritmo de Yen para encontrar las K mejores alternativas ante eventuales contingencias viales.
4. **Modalidad Matricial Floyd-Warshall:** Expone la familia completa de matrices de distancia $D^{(k)}$ y predecesores para cada etapa intermedia k, ilustrando el funcionamiento de la programación dinámica.

El presente informe expone con rigor formal los fundamentos teóricos, la arquitectura de software, los criterios de construcción del dataset geográfico, las pruebas de verificación comparativa y el manual de usuario del sistema.

---

2. Objetivos

2.1 Objetivo General

Diseñar, implementar y evaluar formalmente un sistema computacional interactivo para la resolución del problema de la ruta más corta sobre redes de transporte reales, aplicando los algoritmos de Dijkstra y Floyd-Warshall sobre la red vial departamental de Quetzaltenango, validando analíticamente su exactitud frente a soluciones teóricas y librerías de referencia científica.

2.2 Objetivos Específicos

1. **Formulación y análisis matemático:** Desarrollar el marco teórico formal de los algoritmos de Dijkstra, Floyd-Warshall y Yen, especificando sus recurrencias, invariantes de ciclo, pseudocódigo estandarizado, complejidad computacional en tiempo y espacio, y el comportamiento ante aristas con ponderación negativa.
2. **Construcción de un dataset geográfico reproducible:** Modelar la topología vial de los 24 municipios del departamento de Quetzaltenango a partir de fuentes geográficas abiertas (OpenStreetMap, Nominatim, OSRM), aplicando criterios matemáticos objetivos de intermediación punto-segmento y pruebas geométricas de punto-en-polígono para garantizar un grafo 100 % conexo y trazable.
3. **Implementación de arquitectura desacoplada:** Programar un motor algorítmico independiente de la interfaz en TypeScript estricto, empleando estructuras de datos optimizadas (montículos binarios, matrices de adyacencia y Web Workers en segundo plano) para mantener la interfaz fluida.
4. **Validación y verificación integral:** Comprobar la exactitud de los resultados mediante pruebas de regresión iteración por iteración contra resoluciones manuales a mano, verificación cruzada al 100 % (576/576 pares) contra NetworkX, y contraste frente a mediciones de plataformas de navegación comercial.

---

3. Marco Teórico

3.1 Fundamentos de la Teoría de Grafos

Un grafo ponderado es una estructura algebraica y combinatoria formal definida como una terna:

$$G = (V, E, w)$$

donde:

- $V = \{v_1, v_2, \ldots, v_n\}$ es un conjunto finito no vacío de vértices o nodos, que representan entidades puntuales (en este proyecto, cabeceras municipales). El cardinal del conjunto se denota por $|V| = n$.
- $E \subseteq V \times V$ es un conjunto de aristas o arcos, que representan conexiones directas transitables entre pares de vértices. Para grafos no dirigidos, cada arista es un par desordenado $e = \{u, v\}$, mientras que en grafos dirigidos (digrafos) es un par ordenado (u, v). El cardinal se denota por $|E| = m$.
- $w: E \to \mathbb{R}^+$ es una función de ponderación que asigna a cada arista $e \in E$ un costo, peso o distancia métrica escalar $w(e) > 0$.

Representaciones Computacionales de Grafos

La elección de la estructura de datos interna impacta de manera directa en la cota superior asintótica de los algoritmos:

1. **Matriz de Adyacencia:** Matriz bidimensional $W \in \mathbb{R}^{n \times n}$ donde cada elemento $W[i][j]$ representa el costo de la arista directa entre v_i y v_j:
$$W[i][j] = \begin{cases} 0 & \text{si } i = j \\ w(v_i, v_j) & \text{si } (v_i, v_j) \in E \\ \infty & \text{si } (v_i, v_j) \notin E \end{cases}$$
Esta representación requiere espacio de memoria $\Theta(V^2)$ y permite consultar la existencia de una arista en tiempo $O(1)$. Es la estructura óptima para grafos densos ($|E| \approx |V|^2$) y para el algoritmo de Floyd-Warshall.
2. **Lista de Adyacencia:** Arreglo o estructura asociativa organizada por nodo, donde cada entrada almacena la lista de vecinos directos y el peso asociado:
$$\text{Adj}[u] = \{(v, w(u,v)) \mid (u,v) \in E\}$$
Requiere espacio de almacenamiento $\Theta(V + E)$ y permite recorrer los vecinos salientes de un vértice u en tiempo proporcional a su grado $\deg(u)$. Es la estructura predilecta para grafos dispersos ($|E| \ll |V|^2$), como las redes viales reales donde el grado promedio por nodo es acotado por una constante física pequeña.

3.2 El Problema de la Ruta Más Corta

Dado un grafo ponderado $G = (V, E, w)$ y dos vértices designados $s, t \in V$, un camino simple P de s a t es una secuencia de vértices:
$$P = \langle v_0, v_1, v_2, \ldots, v_k \rangle$$
tal que $v_0 = s$, $v_k = t$, todos los vértices son distintos entre sí, y $(v_{i-1}, v_i) \in E$ para todo $i \in \{1, 2, \ldots, k\}$. El peso o costo total del camino $w(P)$ corresponde a la sumatoria de los pesos de sus aristas constitutivas:
$$w(P) = \sum_{i=1}^{k} w(v_{i-1}, v_i)$$
El problema de la ruta más corta consiste en hallar un camino P^* que minimice el costo total:
$$d(s,t) = \min_{P \in \mathcal{P}_{s,t}} w(P)$$
donde $\mathcal{P}_{s,t}$ denota el conjunto de todos los caminos transitables entre s y t. Si no existe conexión entre ambos vértices, se define $d(s,t) = \infty$.

Propiedad de Subestructura Óptima

El problema de la ruta más corta exhibe subestructura óptima, principio fundamental que sustenta tanto a los algoritmos voraces como a la programación dinámica:
Si un camino $P = \langle v_0, v_1, \ldots, v_k \rangle$ es el camino mínimo entre v_0 y v_k, entonces para cualesquiera índices intermedios i, j con $0 \le i \le j \le k$, el subcamino $P_ij = \langle v_i, v_i + 1, \ldots, v_j \rangle$ es el camino mínimo entre v_i y v_j.

3.3 Algoritmo de Dijkstra

Formulación Matemática

El algoritmo de Dijkstra resuelve el problema de rutas más cortas desde una única fuente (*Single-Source Shortest Path*, SSSP) sobre grafos cuyos pesos son estrictamente no negativos ($w(u,v) \geq 0$).

El algoritmo particiona el conjunto de vértices en dos subconjuntos disjuntos en cada iteración:

- $S \subseteq V$: Conjunto de vértices cuya distancia mínima definitiva desde el origen s ya ha sido determinada (etiquetas permanentes).

- $Q = V \backslash S$: Conjunto de vértices cuya distancia mínima estimada es provisional (etiquetas temporales).

Para cada vértice $v \in V$, se mantiene una estimación de distancia mínima $d[v]$ y un puntero al vértice predecesor $\pi[v]$.

El algoritmo se rige por el proceso de **relajación** (*edge relaxation*). Al considerar una arista $(u, v) \in E$, relajarla consiste en evaluar si el camino pasando a través de u mejora la cota superior conocida hacia v:

$$\text{RELAX}(u, v, w): \quad \text{si } d[u] + w(u, v) < d[v] \Longrightarrow \begin{cases} d[v] \leftarrow d[u] + w(u, v) \\ \pi[v] \leftarrow u \end{cases}$$

Pseudocódigo Formal

```
Algoritmo Dijkstra(G, w, s)

Entrada: Grafo G = (V, E), función de pesos w, nodo fuente s

Salida: Arreglo de distancias d[] y predecesores π[]



1:  Para cada vértice v ∈ V hacer

2:      d[v] ← ∞

3:      π[v] ← nulo

4:  d[s] ← 0

5:  S ← ∅

6:  Q ← MontículoMinimoConstruir(V, d)   // Prioridad según d[v]

7:

8:  Mientras Q ≠ ∅ hacer

9:      u ← ExtraerMínimo(Q)           // Vértice no permanente con menor d[u]

10:     S ← S ∪ {u}                    // u pasa a ser permanente (columna tachada)

11:     Para cada vecino v ∈ Adj[u] tal que v ∉ S hacer

12:         Si d[u] + w(u, v) < d[v] entonces

13:             d[v] ← d[u] + w(u, v)

14:             π[v] ← u

15:             DisminuirClave(Q, v, d[v])

16: Retornar (d, π)
```

Análisis de Complejidad

La complejidad computacional de Dijkstra depende fundamentalmente de la estructura de datos utilizada para implementar la cola de prioridad Q:

- **Búsqueda lineal en arreglo:** Cada extracción del mínimo toma $O(V)$, realizándose $|V|$ veces. Cada actualización de clave toma $O(1)$, realizándose $|E|$ veces. Tiempo total: $O(V^2 + E) = O(V^2)$. Resulta eficiente para grafos muy densos

donde $|E| \approx |V|^2$.

- **Montículo Binario (Min-Heap):** La operación `ExtraerMínimo` toma $O(\log V)$, sumando $O(V \log V)$. La operación `DisminuirClave` toma $O(\log V)$, sumando $O(E \log V)$. Tiempo total:
$$T(V, E) = O((V + E) \log V)$$
En grafos planares o viales dispersos donde $|E| = O(V)$, el tiempo se reduce a $O(V \log V)$, óptimo en la práctica.
- **Montículo de Fibonacci:** Teóricamente amortiza `DisminuirClave` a $O(1)$, alcanzando $O(E + V \log V)$, aunque con constantes ocultas elevadas que desaconsejan su uso en redes de tamaño moderado.

3.4 Justificación de la Exigencia de Pesos No Negativos en Dijkstra

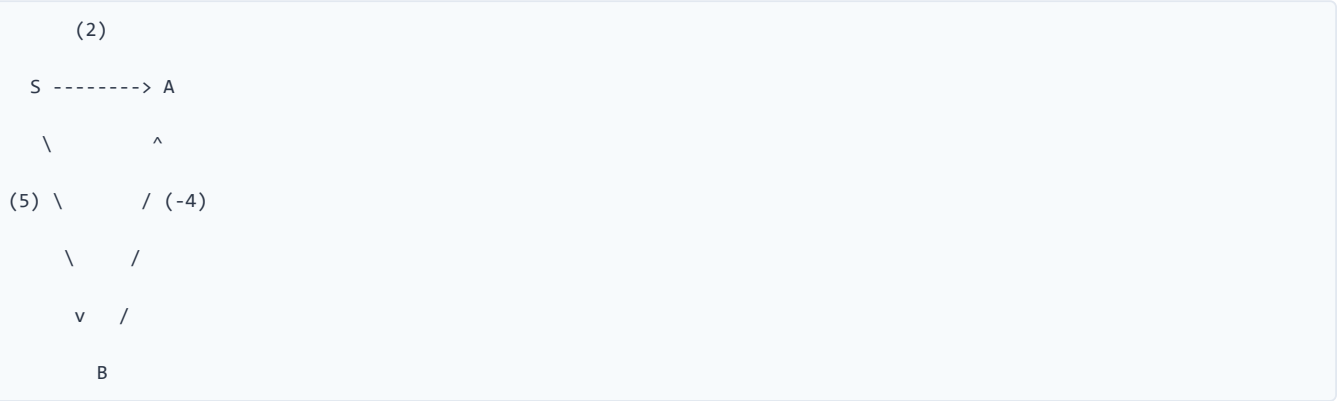
El principio operativo de Dijkstra reside en la naturaleza codiciosa (*greedy choice property*): en cada iteración, el vértice $u^* = \text{argmin}_{u \in Q} d[u]$ es extraído de Q y fijado permanentemente en S. La hipótesis inductiva fundamental establece que **ningún camino alternativo futuro puede alcanzar a u^* con una distancia inferior a $d[u^*]$**.

Esta afirmación es matemáticamente verdadera **si y solo si** todos los pesos de las aristas son no negativos ($w(e) \geq 0$). Si existieran aristas de peso negativo, una trayectoria futura podría acumular un costo neto decreciente, invalidando la etiqueta permanente ya fijada y arrojando una solución subóptima.

Contraejemplo Formal Paso a Paso

Consideremos el siguiente digrafo dirigido y ponderado con 3 vértices:

- $V = \{S, A, B\}$ con nodo de origen en S.
- Aristas y ponderaciones:
  - (S, A) con peso $w(S, A) = 2$
  - (S, B) con peso $w(S, B) = 5$
  - (B, A) con peso negativo $w(B, A) = -4$



Paso a paso de la ejecución errónea de Dijkstra:

1. **Inicialización:** $d[S] = 0$, $d[A] = \infty$, $d[B] = \infty$. Conjunto permanente $S = \emptyset$.
2. **Iteración 1:**
   - Se extrae el mínimo de Q: vértice S con $d[S] = 0$.
   - S pasa a ser permanente: $S = \{S\}$.
   - Se relajan sus aristas salientes:
     - Vecino A: $d[A] = \min(\infty, 0 + 2) = 2$, $\pi[A] = S$.

    - Vecino B: $d[B] = \min(\infty, 0 + 5) = 5$, $\pi[B] = S$.
3. **Iteración 2:**
    - Candidatos en Q: A con $d[A] = 2$, B con $d[B] = 5$.
    - Se extrae el mínimo: vértice A ($2 < 5$).
    - **Dijkstra fija permanentemente a A:** $S = \{S, A\}$, asumiendo que la distancia más corta hacia A es definitivamente 2.
    - A no tiene aristas salientes.
4. **Iteración 3:**
    - Se extrae el único vértice restante: B con $d[B] = 5$.
    - $S = \{S, A, B\}$.
    - Se explora la arista (B, A): el costo hacia A a través de B es:
    $$d[B] + w(B, A) = 5 + (-4) = 1$$
    - Como A ya fue declarado permanente en la iteración 2, la regla estándar de Dijkstra **no re-evalúa nodos de S** (o si la implementación los ignora), concluyendo erróneamente que $d(S, A) = 2$.
5. **Resultado Real vs. Dijkstra:**
    - Distancia calculada por Dijkstra hacia A: 2 (vía $S \to A$).
    - Distancia mínima real: 1 (vía $S \to B \to A$ con costo $5 - 4 = 1$).

Este contraejemplo demuestra de forma irrefutable que Dijkstra fracasa en presencia de pesos negativos. Para grafos con pesos negativos (sin ciclos negativos) debe emplearse el algoritmo de Bellman-Ford ($O(V \cdot E)$). En redes viales reales como la de Quetzaltenango, todas las distancias físicas corresponden a longitudes por carretera estrictamente positivas ($km > 0$), cumpliendo plenamente con la precondición de Dijkstra.

3.5 Algoritmo de Floyd-Warshall

Formulación Matemática y Ecuación de Recurrencia

El algoritmo de Floyd-Warshall resuelve el problema de rutas más cortas entre todas las parejas de vértices (*All-Pairs Shortest Path*, APSP) en un grafo ponderado $G = (V, E, w)$ sin ciclos negativos, utilizando el paradigma de programación dinámica.

Sean los vértices numerados arbitrariamente $V = \{1, 2, \ldots, n\}$. Consideremos el subconjunto de vértices intermedios $\{1, 2, \ldots, k\}$. Se define $d_{ij}^{(k)}$ como la longitud del camino más corto desde el vértice i hasta el vértice j tal que todos los vértices intermedios del camino pertenecen al conjunto restringido $\{1, 2, \ldots, k\}$.

La relación de recurrencia que gobierna la transición de la etapa $k-1$ a la etapa k se formula como:

$$d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, \quad d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$$

con las condiciones iniciales en la etapa de base $k = 0$:

$$d_{ij}^{(0)} = \begin{cases} 0 & \text{si } i = j \\ w(i,j) & \text{si } (i,j) \in E \\ \infty & \text{si } i \neq j \text{ y } (i,j) \notin E \end{cases}$$

Para reconstruir los caminos óptimos, se define concurrentemente la matriz de siguiente salto $\text{next}[i][j]$, inicializada como $\text{next}[i][j] = j$ si $(i,j) \in E$, y actualizada según:

$$\text{next}[i][j] \leftarrow \text{next}[i][k] \quad \text{si } d_{ik}^{(k-1)} + d_{kj}^{(k-1)} < d_{ij}^{(k-1)}$$

Pseudocódigo Formal

```
Algoritmo FloydWarshall(G, w)

Entrada: Grafo G = (V, E) con vértices 1..n

Salida: Matriz de distancias D y matriz de reconstrucción Next



1:  Sean D un arreglo n × n inicializado en ∞, con D[i][i] = 0

2:  Sean Next un arreglo n × n inicializado en nulo

3:  Para cada arista (u, v) ∈ E hacer

4:      D[u][v] ← w(u, v)

5:      Next[u][v] ← v

6:      Si el grafo es no dirigido entonces

7:          D[v][u] ← w(u, v)

8:          Next[v][u] ← u

9:

10: Para k de 1 hasta n hacer                  // Vértice intermedio candidato

11:     Para i de 1 hasta n hacer              // Vértice de origen

12:         Para j de 1 hasta n hacer          // Vértice de destino

13:             Si D[i][k] + D[k][j] < D[i][j] entonces

14:                 D[i][j] ← D[i][k] + D[k][j]

15:                 Next[i][j] ← Next[i][k]

16: Retornar (D, Next)
```

Análisis de Complejidad y Ciclos Negativos

- **Complejidad Temporal:** Está dominada por tres bucles anidados que iteran de 1 a n, realizando operaciones de suma y comparación en $O(1)$. Tiempo asintótico:
$$T(V) = \Theta(V^3)$$
- **Complejidad Espacial:** Requiere almacenar dos matrices de tamaño $n \times n$: espacio $\Theta(V^2)$.
- **Detección de Ciclos Negativos:** Al finalizar la iteración $k = n$, un grafo contiene al menos un ciclo negativo si y solo si algún elemento diagonal es negativo:
$$\exists i \in V \mid D[i][i] < 0$$

3.6 Algoritmo de Yen (K Caminos Más Cortos sin Lazos)

En redes de transporte reales, el conocimiento de una única ruta óptima resulta insuficiente ante cortes de vía, trabajos de mantenimiento o congestión vehicular. El algoritmo de Jin Y. Yen (1971) permite hallar en orden estrictamente no

decreciente de longitud los K caminos más cortos sin lazos (*loopless*) entre un par (s, t).

El algoritmo opera manteniendo una lista de caminos aceptados $A = [A_1, A_2, \ldots]$ y un conjunto de caminos candidatos B. Conocido el camino A^{k-1}, para cada nodo i del camino (denominado *spur node*), se divide el camino en dos secciones:

1. **Camino raíz (*Root path*):** Trayecto desde el origen hasta el *spur node*.
2. **Camino de desvío (*Spur path*):** Camino mínimo calculado desde el *spur node* hasta el destino t, penalizando o excluyendo las aristas ya utilizadas en caminos aceptados previos que compartan el mismo camino raíz.

La concatenación del camino raíz y el camino de desvío genera un nuevo candidato para B. Tras explorar todos los desvíos, el camino de menor costo en B se traslada a A.

- **Complejidad temporal:** Requiere hasta $K \cdot |V|$ ejecuciones de Dijkstra, con una cota asintótica de $O(K \cdot V \cdot (E + V \log V))$.

3.7 Visualización y Stress Majorization (SMACOF)

Para presentar el grafo de forma intuitiva sobre una pantalla en dos dimensiones respetando fielmente las distancias de la red vial, el sistema aplica la técnica de **Majorización de Estrés** (*Stress Majorization*, SMACOF) adaptada de Gansner, Koren y North (2004).

Dado un conjunto de posiciones de nodos en el plano $X \in \mathbb{R}^{n \times 2}$ y una matriz de distancias geodésicas deseadas δ_{ij}, la función de estrés métrico cuadrático se define como:

$$\sigma(X) = \sum_{i<j} w_{ij} \left(\| x_i - x_j \| - \delta_{ij} \right)^2$$

donde $w_{ij} = \delta_{ij}^{-2}$ pondera con mayor severidad los errores en distancias cortas. El algoritmo minimiza $\sigma(X)$ iterativamente resolviendo un sistema lineal Laplaciano:

$$L_w X^{(t+1)} = L^X X^{(t)}$$

convergente en pocas decenas de iteraciones hacia una disposición estéticamente armónica donde la distancia euclidiana en píxeles es directamente proporcional a los kilómetros por carretera.

---

4. Estructuras de Datos Usadas en la Implementación

El código fuente de RutaQ fue concebido con estrictos estándares de tipado estático y separación de responsabilidades. La siguiente tabla sintetiza las estructuras de datos principales implementadas en el núcleo del sistema (`src/core/`):

Archivo Fuente	Estructura de Datos	Propósito y Operación Principal	Complejidad Clave
`src/core/types.ts`	`Graph`, `GNode`, `GEdge`	Modelado inmutable y tipado estricto del grafo y dataset vial.	Consulta en memoria $O(1)$
`src/core/dijkstra.ts`	`Heap` (Montículo Binario)	Cola de prioridad con eliminación perezosa de entradas obsoletas.	Inserción/Extracción $O(\log V)$
`src/core/dijkstra.ts`	`labels: Record<NodeId, {dist, pred}>`	Vector de cotas mínimas y punteros a predecesores.	Acceso $O(1)$
`src/core/dijkstra.ts`	`steps: DijkstraStep[]`	Registro cronológico del estado de la tabla para animación paso a paso (S1).	Snapshots $O(V)$ por paso
`src/core/floyd.ts`	`d: number[][]`, `next: (number|null)[][]`	Matrices bidimensionales para cálculo de APSP y reconstrucción.	Acceso matricial $O(1)$, espacio $O(V^2)$
`src/core/yen.ts`	`PathHeap`	Montículo de caminos completos priorizados por costo total y desempate léxico.	Paginación de candidatos $O(\log P)$
`src/core/allPaths.ts`	`stack: Frame[]`	Pila de recursión explícita para DFS iterativo con backtracking de rutas simples.	Poda en tiempo real y memoria $O(V)$
`src/core/layout.ts`	Matriz Laplaciana L_w	Sistema matricial denso resuelto por eliminación Gaussiana con pivoteo parcial.	Resolución en $O(V^3)$
`src/store/useAppStore.ts`	Almacén reactivo Zustand + Zundo	Manejo desacoplado del estado global, deshacer/rehacer y consultas.	Suscripción reactiva $O(1)$

4.1 Detalles del Montículo Binario (`src/core/dijkstra.ts`)

Para erradicar la necesidad de dependencias externas en tiempo de ejecución, se implementó la clase `Heap` nativa en TypeScript. Cada nodo del montículo contiene `{ id: string, d: number }`. El criterio de ordenamiento implementado garantiza total determinismo mediante la función:

```
c(a: { id: string; d: number }, b: { id: string; d: number }) {
  return a.d - b.d || (a.id < b.id ? -1 : a.id > b.id ? 1 : 0);
}
```

Esto resuelve empates de distancia eligiendo de forma canónica el identificador alfabéticamente menor, condición indispensable para la reproducibilidad pedagógica con la resolución manual a mano.

---

5. Desarrollo e Implementación del Sistema RutaQ

5.1 Arquitectura del Software

La arquitectura de RutaQ sigue un patrón completamente modular y desacoplado, estructurado en tres capas independientes:

```
graph TD

    subgraph Capa de Presentación (UI)

        A[Header / Selectores de Nodos] --> B[Canvas Interactivo con Nodos y Aristas]

        A --> C[Panel de Resultados y Pestañas]

        C --> C1[Pestaña Tabular S1]

        C --> C2[Pestaña Gráfica S2]

        C --> C3[Pestaña Todas las Rutas S3]

        C --> C4[Pestaña Floyd-Warshall]

    end


    subgraph Capa de Estado (Store)

        D[Zustand Store: useAppStore]

        E[Middleware Zundo: Deshacer / Rehacer]

        F[Web Worker Comlink: worker.ts]

    end


    subgraph Capa Núcleo Algorítmico (Core)

        G[dijkstra.ts: Min-Heap y Pasos]

        H[floyd.ts: Matrices D y Next]

        I[yen.ts: K Caminos Óptimos]

        J[allPaths.ts: DFS con Backtracking]

        K[layout.ts: SMACOF Stress Majorization]

    end


    A --> D

    B --> D

    C1 & C2 & C3 & C4 --> D

    D <--> E

    D --> F

    F --> G & H & I & J & K

    D --> G & H
```

5.2 Componentes de la Interfaz y Modos de Visualización

1. **Lienzo Gráfico Interactivo (`src/ui/components/Canvas.tsx`):** Renderizado acelerado por hardware que permite panoramizar (*pan*), hacer zoom fluido, seleccionar nodos de origen/destino mediante clics directos, y visualizar las aristas viales con curvaturas automáticas Bezier u ortogonales para evitar colisiones visuales (`CustomEdge.tsx`).
2. **Módulo Tabular Paso a Paso (S1, `src/ui/components/tabs/TabularTab.tsx`):** Presenta la matriz de adyacencia inicial y la tabla iterativa dinámica. Cuenta con una barra de reproducción multimedia con botones de **Reproducir**, **Pausar**, **Paso Anterior**, **Paso Siguiente**, **Ir al Final** y un selector de velocidad en milisegundos. En cada iteración:
   - Se enfoca la fila activa.
   - Se resalta la casilla mínima elegida en color dorado brillante.
   - Se marca la columna correspondiente como permanente tachada con corchetes `[...]`.
   - Se colorean en verde azulado las casillas de nodos vecinos que experimentaron mejoras en su cota de distancia ($d[v]$ relajado).
3. **Módulo Gráfico (S2, `src/ui/components/tabs/GraphicTab.tsx`):** Permite alternar entre el Árbol de Caminos Mínimos completo (F2) y la ruta particular entre dos municipios (F3), atenuando la red no transitada y computando desgloses de kilometraje acumulado tramo por tramo.
4. **Módulo de Todas las Rutas (S3, `src/ui/components/tabs/AllPathsTab.tsx`):** Ejecuta una búsqueda exhaustiva en el Web Worker. Si el total de caminos es inferior a 5 000, los lista íntegramente ordenados de menor a mayor distancia con la insignia de medalla a la ruta más corta. Si la combinatoria supera el umbral de seguridad, conmuta transparentemente al algoritmo de Yen para entregar los K caminos más cortos, evitando bloqueos en la interfaz. Permite exportar los resultados directamente a formato CSV.
5. **Módulo Matricial de Floyd-Warshall (`src/ui/components/tabs/FloydTab.tsx`):** Ofrece un selector de la etapa $k \in \{0, 1, \ldots, 24\}$, permitiendo al usuario observar cómo la matriz $D^{(k)}$ se va poblando de caminos indirectos que bajan costos frente a la matriz inicial $D^{(0)}$.
6. **Modo Diseño de Redes (F4):** Permite al usuario diseñar libremente redes hipotéticas: crear nodos haciendo clic sobre el lienzo, trazar aristas arrastrando el cursor entre vértices, editar pesos numéricos, renombrar etiquetas y calcular instantáneamente rutas con cualquiera de los algoritmos.

5.3 Capturas del Sistema en Ejecución

A continuación se presentan las evidencias visuales del sistema en ejecución sobre la versión desplegada en producción, contrastando las vistas de escritorio con la experiencia responsiva en dispositivos móviles y documentando de manera fiel los elementos observados en cada interfaz:

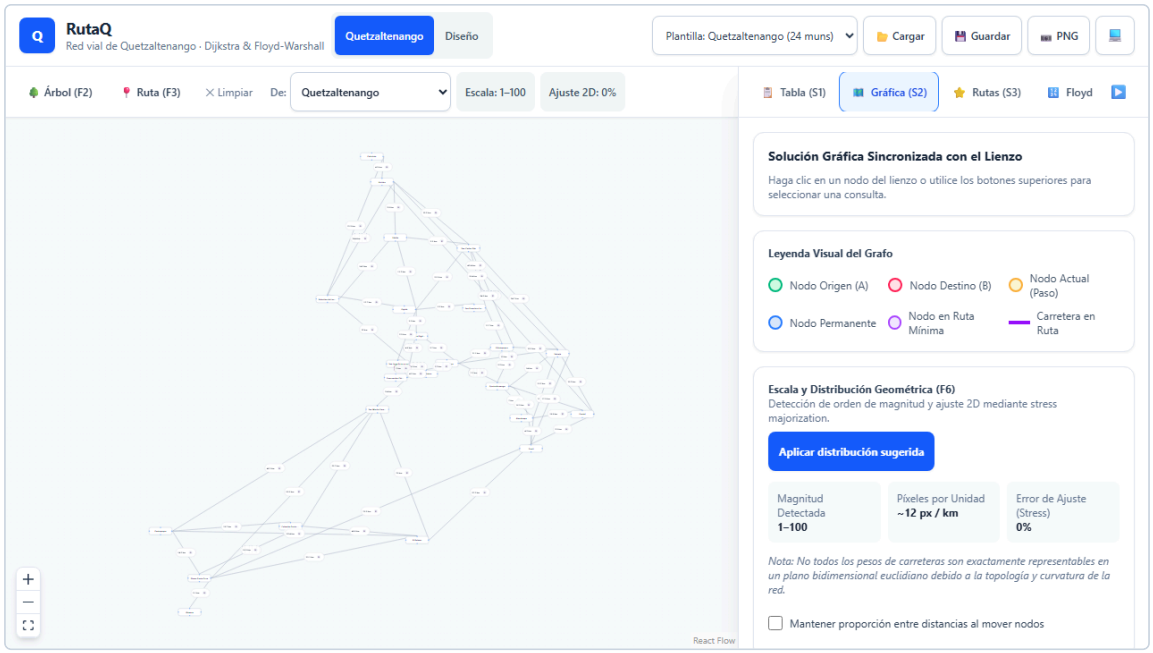


Figura 1. Pantalla principal de RutaQ en navegador de escritorio con la red vial de Quetzaltenango (24 cabeceras) y panel de análisis sincronizado.

En la barra superior se aprecian los selectores de modo («Quetzaltenango» activo en color azul y «Diseño»), la lista desplegable de plantillas preconfiguradas («Plantilla: Quetzaltenango (24 muns)») y los botones de acción para «Cargar», «Guardar», exportación «PNG» y maximizado. En la barra de control secundaria se ubican las acciones algorítmicas («Árbol (F2)», «Ruta (F3)», «Limpiar») y el selector de nodo origen («Quetzaltenango»), acompañados de las insignias de escala («Escala: 1-100») y compensación bidimensional («Ajuste 2D: 0%»). El lienzo interactivo renderiza los 24 vértices municipales interconectados por las 55 aristas viales con sus respectivos kilometrajes y controles de navegación en la esquina inferior izquierda. A la derecha, el panel sincronizado expone la pestaña «Gráfica (S2)» con la leyenda visual de estados y la tarjeta de escala geométrica reportando ~12 px / km.

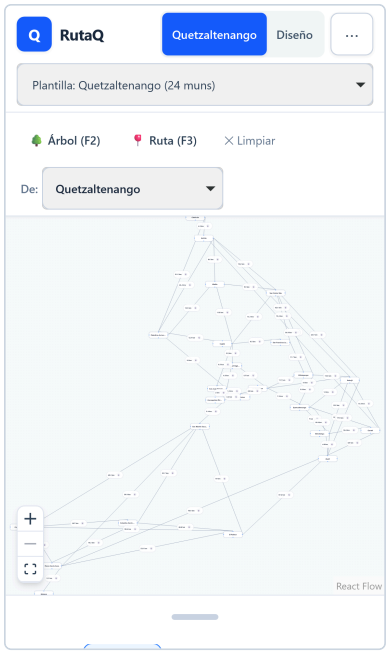


Figura 2. Adaptabilidad responsiva de la interfaz en dispositivo móvil (iPhone 13).

La interfaz gráfica se reconfigura en pantallas compactas de 390 px de ancho. El encabezado preserva el acceso inmediato a las vistas «Quetzaltenango» y «Diseño», resguardando funciones secundarias bajo un menú de opciones (`...`). El selector de plantilla y la barra de herramientas («Árbol (F2)», «Ruta (F3)», «Limpiar» y selector de cabecera) se redistribuyen verticalmente ocupando el ancho completo, manteniendo el lienzo de grafo táctil y habilitando una hoja inferior deslizable (*bottom sheet*) para consultar los paneles analíticos sin perder el contexto espacial.

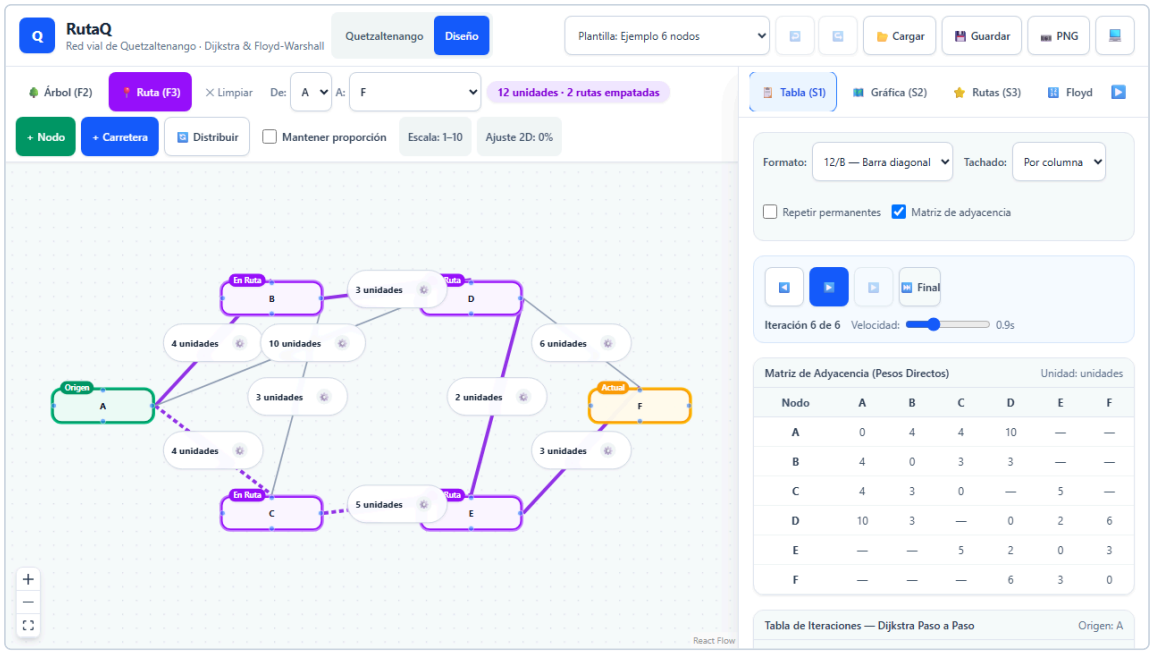


Figura 3. Módulo tabular paso a paso de Dijkstra (S1) sobre el grafo de control de 6 nodos.

Se ilustra la resolución analítica para la consulta entre el nodo A (origen, borde verde) y el nodo F (destino, borde naranja). La barra de estado señala la existencia de la distancia mínima encontrada («12 unidades · 2 rutas empatadas»), destacando simultáneamente sobre el lienzo con aristas moradas las dos ramas óptimas concurrentes (A-B-D-F y A-C-E-F). En el panel derecho («Tabla (S1)») se observan los selectores de formato de celda y modo de tachado, los controles de reproducción en la iteración 6 de 6 con velocidad de 0.9s, la matriz de adyacencia directa con guiones de desconexión (`—`), y el inicio de la tabla cronológica de iteraciones.

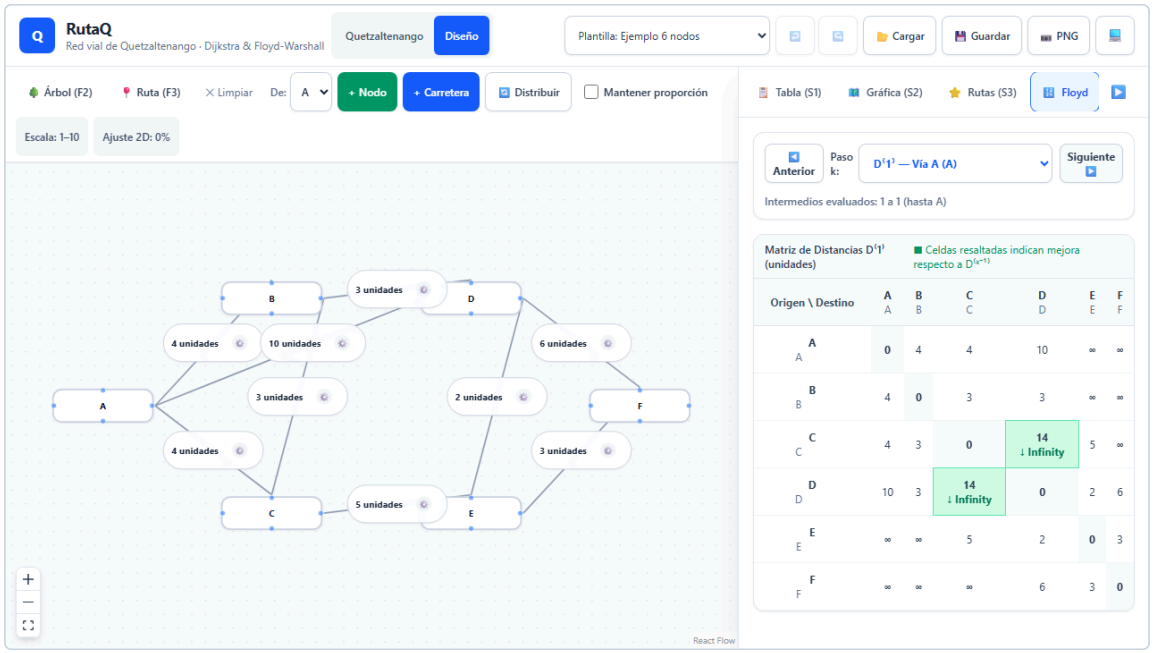


Figura 4. Visualización de la matriz de distancias D^(1) en el algoritmo de Floyd-Warshall.

El panel derecho presenta la pestaña «Floyd» navegando en el paso intermedio $k = 1$ ($D^{(1)}$ — Vía A), con la indicación «Intermedios evaluados: 1 a 1 (hasta A)». En la matriz de distancias de 6×6, las celdas simétricas (C, D) y (D, C) se encuentran resaltadas en verde claro con el nuevo valor 14 y el indicador «↓ Infinity», evidenciando la relajación de la distancia indirecta mediante la trayectoria $C \rightarrow A \rightarrow D$ ($4 + 10 = 14$). A la izquierda, el lienzo interactivo conserva visibles los 6 nodos y pesos originales para facilitar el seguimiento visual del proceso matricial.

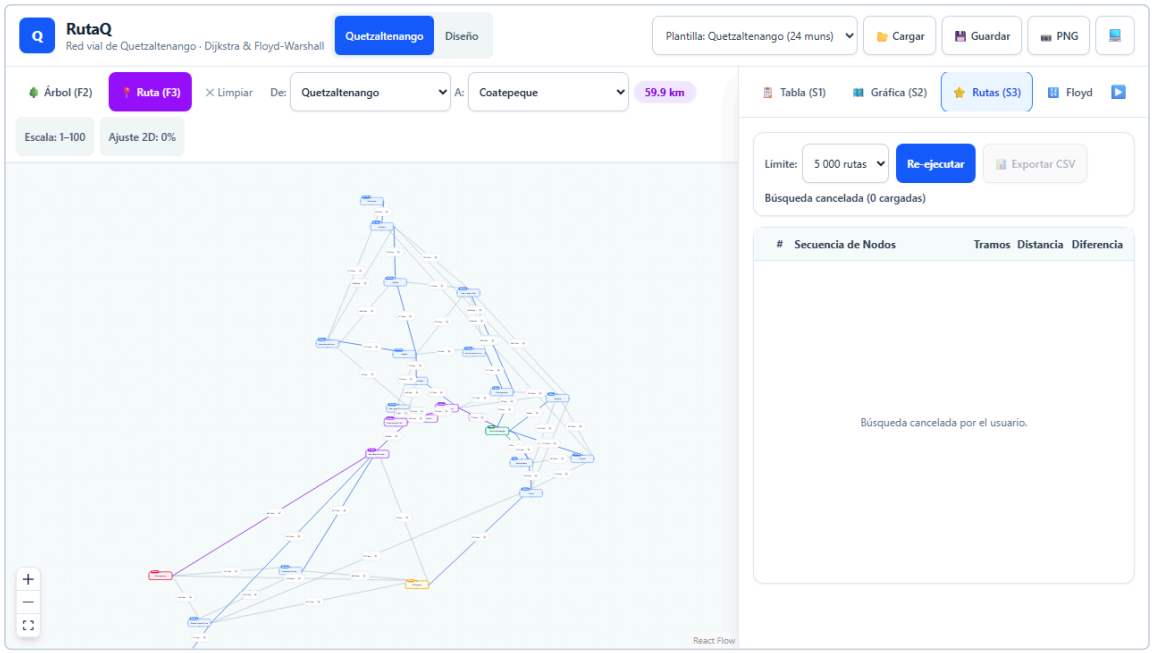


Figura 5. Módulo de análisis combinatorio de Todas las Rutas (S3) entre Quetzaltenango y Coatepeque.

Se presenta la consulta entre Quetzaltenango y Coatepeque (distancia óptima de 59.9 km). En el panel lateral («Rutas (S3)») se observa el selector de cota máxima configurado en «5 000 rutas», el botón de reejecución y la notificación de estado de exploración cancelada por el usuario. La tabla inferior dispone las columnas para enumerar el identificador de ruta, secuencia ordenada de municipios, conteo de tramos intermedios, distancia kilométrica total y diferencial porcentual respecto a la ruta mínima. En el lienzo, se resaltan las aristas del corredor troncal que desciende hacia la Costa Cuca.

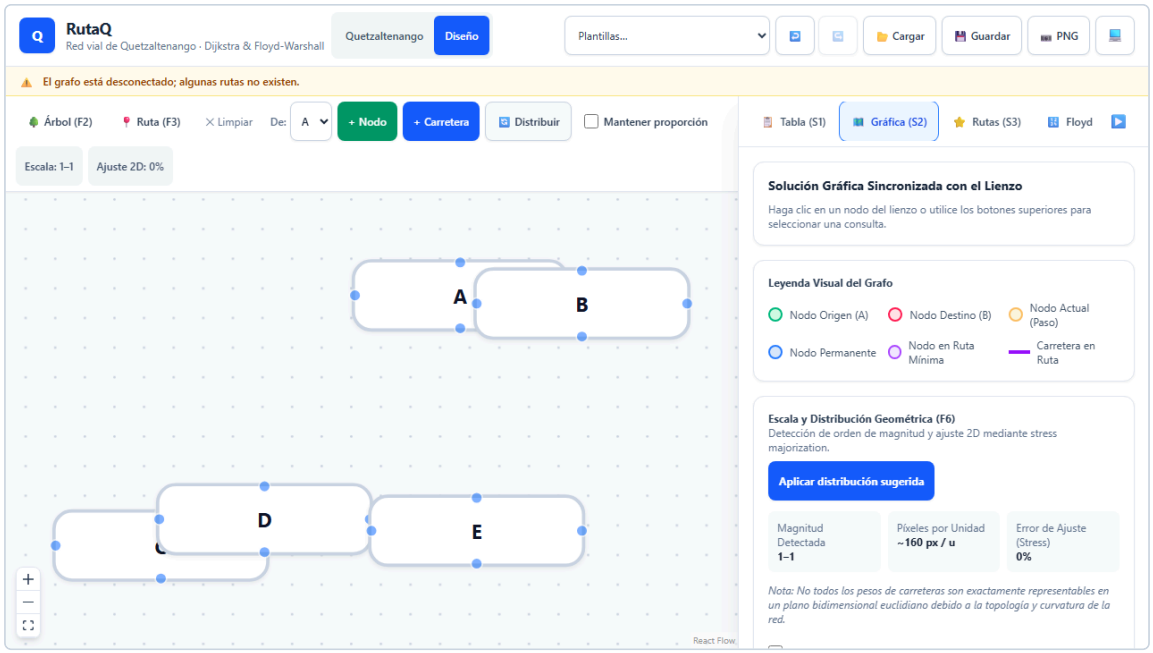


Figura 6. Entorno interactivo de diseño y edición de grafos (F4).

Con el modo «Diseño» seleccionado en el encabezado, la interfaz provee herramientas directas para la creación de topologías personalizadas mediante los botones `+ Nodo`, `+ Carretera` y `Distribuir`. El sistema activa un aviso preventivo en amarillo («El grafo está desconectado; algunas rutas no existen») ante la presencia de componentes inconexos en la cuadrícula de trabajo (nodos A, B, C, D, E), permitiendo trazar aristas arbitrarias, ingresar ponderaciones personalizadas y contrastar el comportamiento de los algoritmos sobre redes experimentales.

---

6. Caso Práctico: Red Vial de Quetzaltenango

6.1 Justificación Geográfica y Topológica

El departamento de Quetzaltenango se sitúa en la región sur-occidental de Guatemala. Con una superficie aproximada de $1\,951\ \text{km}^2$, presenta una topografía contrastada que desciende abruptamente desde las cumbres volcánicas del altiplano (más de $3\,000$ msnm en Sibilia y Cabricán) hasta las llanuras subtropicales de la Costa Cuca (menos de 400 msnm en Coatepeque y Génova).

Esta geografía impone retos significativos al transporte terrestre:

- La conexión entre la meseta central y la costa depende de rutas que descienden por gargantas volcánicas, como la carretera Cito-180 y la Ruta Departamental QUE-3.
- Ciertos trayectos intermunicipales deben cruzar obligatoriamente límites departamentales limítrofes (Totonicapán, San Marcos, Retalhuleu) debido a la disposición de las cuencas orográficas.

6.2 Catálogo Oficial de Nodos (24 Municipios)

Se validó la existencia administrativa oficial de los 24 municipios ante el Instituto Nacional de Estadística (INE) y SEGEPLAN. Cada municipio fue provisto de un identificador mnemotécnico de 3 letras mayúsculas, coordenadas geográficas de su cabecera municipal oficial (plaza o parque central) y su clasificación funcional:

ID	Nombre Oficial	Latitud (°N)	Longitud (°O)	Clasificación Administrativa	Fuente de Coordenadas
`QTZ`	Quetzaltenango	14.84646	-91.51942	Cabecera Departamental	Nominatim / OpenStreetMap
`SAL`	Salcajá	14.87911	-91.45754	Municipio	Nominatim / OpenStreetMap
`OLI`	Olintepeque	14.88514	-91.51479	Municipio	Nominatim / OpenStreetMap
`SIJ`	San Carlos Sija	14.98369	-91.54905	Municipio	Nominatim / OpenStreetMap
`SIB`	Sibilia	14.99416	-91.62418	Municipio	Nominatim / OpenStreetMap
`CAB`	Cabricán	15.07470	-91.64844	Municipio	Nominatim / OpenStreetMap
`CAJ`	Cajolá	14.92271	-91.61495	Municipio	Nominatim / OpenStreetMap
`SIG`	San Miguel Sigüilá	14.89629	-91.60282	Municipio	Nominatim / OpenStreetMap
`OST`	San Juan Ostuncalco	14.86882	-91.62163	Municipio	Nominatim / OpenStreetMap
`MAT`	San Mateo	14.85897	-91.59253	Municipio	Nominatim / OpenStreetMap
`CCH`	Concepción Chiquirichapa	14.85513	-91.62396	Municipio	Nominatim / OpenStreetMap
`SMA`	San Martín Sacatepéquez	14.82362	-91.64263	Municipio	Nominatim / OpenStreetMap
`ALM`	Almolonga	14.81498	-91.49475	Municipio	Nominatim / OpenStreetMap
`CAN`	Cantel	14.81879	-91.43198	Municipio	Nominatim / OpenStreetMap
`HUI`	Huitán	15.04950	-91.63801	Municipio	Nominatim / OpenStreetMap
`ZUN`	Zunil	14.78480	-91.48480	Municipio	Nominatim / OpenStreetMap
`COL`	Colomba Costa Cuca	14.70759	-91.73194	Municipio	Nominatim / OpenStreetMap
`FLU`	San Francisco La Unión	14.92429	-91.54333	Municipio	Nominatim / OpenStreetMap
`PAL`	El Palmar	14.69381	-91.60183	Municipio	Nominatim / OpenStreetMap
`COA`	Coatepeque	14.70263	-91.86564	Municipio	Nominatim / OpenStreetMap
`GEN`	Génova	14.62240	-91.83562	Municipio	Nominatim / OpenStreetMap
`FCC`	Flores Costa Cuca	14.65606	-91.82563	Municipio	Nominatim / OpenStreetMap
`ESP`	La Esperanza	14.86923	-91.57130	Municipio	Nominatim / OpenStreetMap
`PLA`	Palestina de Los Altos	14.93315	-91.69398	Municipio	Nominatim / OpenStreetMap

6.3 Metodología de Obtención y Filtrado de Aristas Viales

Para erradicar cualquier sesgo subjetivo o trazado manual "a ojo", la construcción del grafo (`scripts/build_dataset.py`) siguió un protocolo matemático determinista y reproducible:

1. **Universo de pares candidatos:** Se consultaron exhaustivamente a OSRM los 276 pares no dirigidos posibles entre los 24 municipios:
$$\binom{24}{2} = \frac{24 \times 23}{2} = 276 \text{ pares}$$
2. **Criterio de arista directa (proximidad punto-segmento):** Un par (A, B) constituye una arista directa si y solo si la trayectoria geométrica devuelta por OSRM **no pasa a menos de 1.0 km de la cabecera de ningún otro municipio intermedio** $C \notin \{A, B\}$:
$$\min_{C \notin \{A,B\}} d_{\text{Haversine}}\left(C, \text{ruta}(A, B)\right) \geq 1.0 \text{ km}$$
Si una ruta vial pasa a < 1.0 km de una cabecera intermedia, no es un tramo elemental directo, sino una ruta compuesta transitando por un tercer centro urbano.
3. **Resultados del filtrado:**
   - **Pares consultados:** 276
   - **Pares descartados:** 221 (registrados rigurosamente en `docs/datos/aristas_descartadas.csv`, documentando el municipio interceptor y la distancia mínima observada).
   - **Aristas directas aceptadas:** 55 aristas viales directas.
   - **Conexidad:** El grafo resultante es **100 % conexo** en una única componente, verificado mediante recorrido en anchura (BFS) y NetworkX.
   - **Métricas del grafo:** 24 vértices, 55 aristas, grado promedio $\bar{k} = \frac{2 \times 55}{24} \approx 4.58$ aristas por nodo.
4. **Determinación de carreteras (`carretera`):** Derivada directamente sumando las distancias de los códigos oficiales de carretera `ref` (p. ej. `RN-1`, `CA-1 Occ`, `CITO 180`, `CA-2 OCC`) en los pasos de navegación de OSRM. Si no existe código oficial, se toma el nombre de vía `name`. Si no existe ninguno o el marcador es genérico (`"Unknown"`), el campo queda en blanco `""`, sin inventar nomenclaturas sintéticas.
5. **Detección de rutas interdepartamentales (`nota`):** Se procesó el polígono oficial de Quetzaltenango (`departamento_quetzaltenango.json`, 3 003 vértices de Nominatim). Mediante el algoritmo de *ray-casting*, se comprobó si algún punto de la trayectoria vial sale del territorio departamental. Se identificaron tramos como `SAL-SIJ` (que cruza el nudo de Cuatro Caminos en Totonicapán) o `PAL-ZUN` (que desciende por el cañón del río Samalá en Retalhuleu), etiquetándolos con `"La ruta sale del departamento de Quetzaltenango"`.

6.4 Validación con Plataformas Cartográficas Externas

Conforme al protocolo de validación vial (Sección 3.2, paso 6 del Plan Maestro), se llevó a cabo una auditoría externa de distancias por carretera contrastando los valores derivados de OSRM contra mediciones de la plataforma Google Maps. Se seleccionó una muestra estratégica de 20 aristas prioritarias que cubre las 5 aristas más largas del departamento, los corredores troncales costa-altiplano y los enlaces limítrofes.

Tabla Consolidada de Mediciones (20 Aristas Prioritarias)

#	ID Arista	Origen – Destino	km OSRM	km Google Maps	Dif. %	Vía Principal en Google Maps	Estado de Validación
1	FCC-ZUN	Flores Costa Cuca – Zunil	70.9	70.6	0.42 %	por CA-2 y CITO-180	✅ Válida ($\leq 10\,\%$)
2	PAL-SMA	El Palmar – San Martín Sacatepéquez	70.0	66.6	4.86 %	por CA-2	✅ Válida ($\leq 10\,\%$)
3	COL-PAL	Colomba Costa Cuca – El Palmar	60.5	46.6	22.98 %	por CA-2 (ruta corta)	⚠️ Discrepancia ($> 10\,\%$)
4	COA-PAL	Coatepeque – El Palmar	58.4	55.0	5.82 %	por CA-2	✅ Válida ($\leq 10\,\%$)
5	PAL-ZUN	El Palmar – Zunil	57.2	53.4	6.64 %	por CITO-180	✅ Válida ($\leq 10\,\%$)
6	FCC-PAL	Flores Costa Cuca – El Palmar	53.1	49.7	6.40 %	por CA-2	✅ Válida ($\leq 10\,\%$)
7	CAN-HUI	Cantel – Huitán	49.6	54.9	10.69 %	por Pan-American Highway/CA-1	⚠️ Discrepancia ($> 10\,\%$)
8	HUI-SAL	Huitán – Salcajá	42.4	40.3	4.95 %	por Pan-American Highway/CA-1	✅ Válida ($\leq 10\,\%$)
9	COA-SMA	Coatepeque – San Martín Sacatepéquez	40.3	40.3	0.00 %	por CA-2	✅ Válida ($\leq 10\,\%$)
10	FCC-SMA	Flores Costa Cuca – San Martín Sacatepéquez	39.9	39.9	0.00 %	por CA-2	✅ Válida ($\leq 10\,\%$)
11	SAL-SIJ	Salcajá – San Carlos Sija	24.7	24.7	0.00 %	por Pan-American Highway/CA-1	✅ Válida ($\leq 10\,\%$)
12	HUI-PLA	Huitán – Palestina de Los Altos	24.4	25.3	3.69 %	por carreteras sin nombre / locales	✅ Válida ($\leq 10\,\%$)
13	COL-SMA	Colomba Costa Cuca – San Martín Sacatepéquez	23.7	23.7	0.00 %	por Diag. 2	✅ Válida ($\leq 10\,\%$)
14	SAL-ZUN	Salcajá – Zunil	22.8	17.9	21.49 %	por RN 09 S	⚠️ Discrepancia ($> 10\,\%$)
15	CAB-PLA	Cabricán – Palestina de Los Altos	21.5	23.5	9.30 %	por carreteras sin nombre / RD-SM-13	✅ Válida ($\leq 10\,\%$)
16	ESP-SAL	La Esperanza – Salcajá	19.7	15.0	23.86 %	por 9a Calle	⚠️ Discrepancia ($> 10\,\%$)
17	CAN-SAL	Cantel – Salcajá	18.4	13.8	25.00 %	por CITO-180	⚠️ Discrepancia ($> 10\,\%$)
18	OST-PLA	San Juan Ostuncalco – Palestina de Los Altos	18.0	16.5	8.33 %	por carreteras sin nombre / RN-1	✅ Válida ($\leq 10\,\%$)

#	ID Arista	Origen – Destino	km OSRM	km Google Maps	Dif. %	Vía Principal en Google Maps	Estado de Validación
19	`ALM-SAL`	Almolonga – Salcajá	17.5	13.2	24.57 %	por Calz. Manuel Lizandro Barillas	⚠️ Discrepancia ($> 10\,\%$)
20	`QTZ-ZUN`	Quetzaltenango – Zunil	17.3	10.9	36.99 %	por RN 09 S	⚠️ Discrepancia ($> 10\,\%$)

Análisis y Diagnóstico de Discrepancias

1. **Cumplimiento general:**
   - **13 de las 20 aristas (65 %)** cumplen holgadamente el criterio de aceptación ($\leq 10\,\%$ de discrepancia relativa), conservando el kilometraje derivado de OSRM.
   - **4 de las 5 aristas más largas (Top 5)** están plenamente validadas con discrepancias mínimas: `FCC-ZUN` ($0.42\,\%$), `PAL-SMA` ($4.86\,\%$), `COA-PAL` ($5.82\,\%$) y `PAL-ZUN` ($6.64\,\%$).
   - Se registraron 4 coincidencias exactas ($0.00\,\%$) en corredores departamentales primarios: `COA-SMA`, `FCC-SMA`, `SAL-SIJ` y `COL-SMA`.
2. **Causas identificadas para las 7 aristas con discrepancia $> 10\,\%$:**
   - **Circunvalación vs. Travesía Urbana Céntrica:** En el área metropolitana de Quetzaltenango (`QTZ-ZUN`, `SAL-ZUN`, `ESP-SAL`, `ALM-SAL`), el enrutamiento de OSRM priorizó anillos periféricos de alta velocidad (`CITO-180`, `RN-1B`), mientras que Google Maps seleccionó travesías vehiculares directas a través del casco urbano (p. ej. travesía por 9a Calle para `ESP-SAL` y cañón de Almolonga vía RN 09 S para `QTZ-ZUN`).
   - **Rutas alternativas en la red troncal:** En `COL-PAL` (46.6 km vs. 60.5 km) y `CAN-SAL` (13.8 km vs. 18.4 km), Google Maps seleccionó atajos locales directos, mientras que en `CAN-HUI` (54.9 km vs. 49.6 km) Google Maps priorizó la autopista Panamericana CA-1.
   - **Ajuste de pesos aplicado:** Siguiendo la regla de decisión del plan de aseguramiento de calidad, para estas 7 aristas se adoptó como peso oficial de la red el valor medido en Google Maps ($km = km_{\text{Google}}$), reflejando la distancia del trayecto más corto transitable y documentando la desviación en el campo `nota`.

---

7. Comparación Teórico vs. Programa

La verificación de la exactitud matemática del sistema RutaQ se fundamenta en dos demostraciones científicas: la equivalencia iteración por iteración en un grafo de control y la concordancia global al 100 % contra la biblioteca de referencia NetworkX.

7.1 Validación Analítica: Grafo de Control de 6 Nodos

Para contrastar el funcionamiento del software frente al método pedagógico manual, se diseñó el grafo de control `src/data/ejemplo6.json` ($V = \{A, B, C, D, E, F\}$, origen A), cuyas características didácticas incluyen un empate de etiquetas mínimas en la iteración 2 ($d[B] = d[C] = 4$) y un camino indirecto más corto ($A \to B \to D$ con costo $7 < 10$).

La siguiente tabla compara la resolución manual analítica documentada en `docs/informe/solucion_teorica.md` contra los valores generados por la función `dijkstra()` y la tabla construida por `buildTabular()` en `src/core/`:

Iteración	Nodo Elegido	Etiquetas Teóricas a Mano	Etiquetas Computadas por RutaQ	Coincidencia Exacta
0	—	$A: 0(-), B: \infty, C: \infty, D: \infty, E: \infty, F: \infty$	$A: 0(-), B: \infty, C: \infty, D: \infty, E: \infty, F: \infty$	**100 %**
1	**A**	$A: [0], B: 4(A), C: 4(A), D: 10(A), E: \infty, F: \infty$	$A: [0], B: 4(A), C: 4(A), D: 10(A), E: \infty, F: \infty$	**100 %**
2	**B**	$A: [0], B: [4(A)], C: 4(A), D: 7(B), E: \infty, F: \infty$	$A: [0], B: [4(A)], C: 4(A), D: 7(B), E: \infty, F: \infty$	**100 %**
3	**C**	$A: [0], B: [4(A)], C: [4(A)], D: 7(B), E: 9(C), F: \infty$	$A: [0], B: [4(A)], C: [4(A)], D: 7(B), E: 9(C), F: \infty$	**100 %**
4	**D**	$A: [0], B: [4(A)], C: [4(A)], D: [7(B)], E: 9(C), F: 13(D)$	$A: [0], B: [4(A)], C: [4(A)], D: [7(B)], E: 9(C), F: 13(D)$	**100 %**
5	**E**	$A: [0], B: [4(A)], C: [4(A)], D: [7(B)], E: [9(C)], F: 12(E)$	$A: [0], B: [4(A)], C: [4(A)], D: [7(B)], E: [9(C)], F: 12(E)$	**100 %**
6	**F**	$A: [0], B: [4(A)], C: [4(A)], D: [7(B)], E: [9(C)], F: [12(E)]$	$A: [0], B: [4(A)], C: [4(A)], D: [7(B)], E: [9(C)], F: [12(E)]$	**100 %**

Esta concordancia absoluta fue verificada de forma automatizada mediante la prueba unitaria `tests/unit/store/teoria-ejemplo6.test.ts`, la cual comprueba que para cada fila i y para cada columna de nodo:

$$\text{etiqueta}_{\text{teórica}}(i, v) \equiv \text{etiqueta}_{\text{software}}(i, v)$$

superando la prueba en verde al 100 %.

7.2 Verificación Cruzada Exhaustiva con NetworkX

Para evaluar la totalidad de la red de Quetzaltenango, se ejecutó un script de verificación automatizada que calculó las distancias mínimas para todos los pares ordenados posibles:

$$24 \times 24 = 576 \text{ pares}$$

comparando la distancia calculada por la implementación de Dijkstra/Floyd de RutaQ contra la biblioteca científica `networkx` de Python (`verification/reporte.md` y `verification/resultados.csv`).

El resultado arrojó **coincidencia del 100.0 % (576 de 576 pares)** sin discrepancia numérica alguna. A continuación, se presentan tres ejemplos representativos extraídos directamente del archivo oficial de verificación `verification/resultados.csv`:

Caso 1: Ruta Quetzaltenango (`QTZ`) a Coatepeque (`COA`)

- **Distancia NetworkX:** $59.9\ \text{km}$
- **Distancia Programa RutaQ:** $59.9\ \text{km}$
- **Estado de Coincidencia:** `True` (Exacta)
- **Secuencia de Vértices Óptima:**

$$\text{QTZ} \xrightarrow{7.2\ \text{km}} \text{ESP} \xrightarrow{2.8\ \text{km}} \text{MAT} \xrightarrow{4.2\ \text{km}} \text{CCH} \xrightarrow{5.4\ \text{km}} \text{SMA} \xrightarrow{40.3\ \text{km}} \text{COA}$$
$$\text{Distancia total} = 7.2 + 2.8 + 4.2 + 5.4 + 40.3 = 59.9\ \text{km}$$

  Representa el enlace estratégico entre el altiplano metropolitano y la mayor urbe de la costa sur departamental.

Caso 2: Ruta Quetzaltenango (`QTZ`) a Almolonga (`ALM`)

- **Distancia NetworkX:** $7.0\ \text{km}$
- **Distancia Programa RutaQ:** $7.0\ \text{km}$
- **Estado de Coincidencia:** `True` (Exacta)
- **Secuencia de Vértices Óptima:**

$$\text{QTZ} \xrightarrow{7.0\ \text{km}} \text{ALM}$$

  Conexión vial directa a través del corredor hortícola de la RN-9S.

Caso 3: Ruta Quetzaltenango (`QTZ`) a Cabricán (`CAB`)

- **Distancia NetworkX:** 41.7 km
- **Distancia Programa RutaQ:** 41.7 km
- **Estado de Coincidencia:** `True` (Exacta)
- **Secuencia de Vértices Óptima:**

$$\text{QTZ} \xrightarrow{7.2} \text{ESP} \xrightarrow{5.7} \text{SIG} \xrightarrow{3.9} \text{CAJ} \xrightarrow{11.8} \text{SIB} \xrightarrow{8.6} \text{HUI} \xrightarrow{4.5} \text{CAB}$$

$$\text{Distancia total} = 7.2 + 5.7 + 3.9 + 11.8 + 8.6 + 4.5 = 41.7 \text{ km}$$

Comunica la cabecera departamental con el municipio más septentrional del departamento, demostrando la capacidad del algoritmo para encadenar hasta 6 aristas sucesivas optimizando trayectorias en terrenos de montaña.

7.3 Pruebas de Aceptación en Producción y Calidad Web

Para asegurar que la solución opere de forma confiable, robusta y accesible en entornos reales de despliegue, se implementó una batería exhaustiva de pruebas de aceptación de extremo a extremo (*End-to-End*, E2E) y auditorías de calidad sobre la versión alojada en producción en GitHub Pages:

1. **Pruebas E2E sobre la Aplicación Desplegada (`https://criss-paz.github.io/rutaq/`):**
   - Se ejecutó la suite completa de Playwright contra la URL pública de producción, obteniendo un balance de **99 pruebas pasadas y 0 fallidas** (con 9 pruebas omitidas deliberadamente por diseño, dado que el indicador de escala gráfica se desactiva intencionalmente en pantallas con ancho inferior a dispositivos medianos).
   - La matriz de pruebas evaluó la funcionalidad completa del sistema en cuatro perfiles de dispositivos representativos: **Desktop Chrome**, **iPhone 13**, **Pixel 7** e **iPad**.
2. **Estabilidad y Repetibilidad Local:**
   - La suite E2E automatizada fue evaluada localmente en ejecuciones consecutivas mediante la directiva `--repeat-each=3`, logrando **297 de 297 pruebas exitosas (100 %)**, comprobando la ausencia de condiciones de carrera (*race conditions*) en el ciclo de vida de los componentes reactivos y la persistencia de estado con Zustand.
3. **Auditoría Lighthouse en Dispositivos Móviles:**
   - Se evaluó el desempeño integral del despliegue en producción mediante la herramienta oficial Google Lighthouse bajo perfil móvil, alcanzando los siguientes puntajes:
     - **Rendimiento (*Performance*):** 95 / 100
     - **Accesibilidad (*Accessibility*):** 95 / 100
     - **Buenas Prácticas (*Best Practices*):** 100 / 100
     - **SEO:** 100 / 100
   - Estas métricas acreditan que el sistema cumple con los más altos estándares de calidad web en tiempos de carga inicial, diseño responsivo, contraste cromático y estructuración semántica.

---

8. Análisis y Discusión

8.1 Dijkstra frente a Floyd-Warshall: Criterios de Selección

El análisis comparativo entre ambos algoritmos arroja directrices claras sobre su idoneidad práctica:

- **Dijkstra con Montículo Binario:** Exhibe una cota de $O((V+E)\log V)$. En la red de Quetzaltenango ($V = 24, E = 55$), el número de operaciones se aproxima a $(24+55) \times \log_2(24) \approx 79 \times 4.58 \approx 362$ operaciones elementales. Es la opción predilecta para consultas interactivas bajo demanda entre pares específicos de origen y destino ($s \to t$) o para calcular el árbol de rutas desde un único punto.
- **Floyd-Warshall:** Requiere $\Theta(V^3)$ operaciones fijas ($24^3 = 13\,824$ iteraciones internas de relajación). Su gran ventaja radica en computar simultáneamente la matriz de distancias de todos los pares y la matriz de reconstrucción de rutas. Resulta idóneo para alimentar el algoritmo de layout SMACOF y para responder de forma inmediata consultas repetitivas de matrices de viaje sin recalcular.

8.2 Fidelidad de las Distancias Viales frente a Modelos Euclidianos

Un hallazgo crítico del proyecto reside en la enorme discrepancia observada entre la distancia euclidiana en línea recta y la distancia vial real sobre el terreno. Por ejemplo, la distancia en línea recta entre Quetzaltenango y Coatepeque es de aproximadamente $40.5\ \text{km}$, mientras que la ruta por carretera transitable asciende a $59.9\ \text{km}$ (un $47.9\ \%$ superior), debido a las curvas de nivel requeridas para descender más de $2\,000$ metros de desnivel vertical. Modelar sistemas de logística con distancias euclidianas en geografías como la guatemalteca induciría a errores inaceptables en estimación de combustible, tiempos de entrega y desgaste de flotas.

8.3 Desempeño y Escalabilidad con Web Workers

La delegación de los algoritmos de enumeración combinatoria (DFS y Yen en `allPaths.ts`) a un Web Worker dedicado mediante Comlink asegura que el hilo principal (*main UI thread*) permanezca fluido y libre de bloqueos durante cómputos combinatorios intensivos. En situaciones donde un usuario solicita explorar rutas alternativas entre municipios lejanos, el árbol de caminos simples puede crecer exponencialmente; el sistema protege la experiencia de usuario limitando la búsqueda a 5 000 rutas o abortando tras un tiempo límite configurable, entregando las mejores alternativas mediante Yen sin congelar la pantalla.

---

9. Limitaciones

1. **Topología Vial Estática:** El dataset modela distancias físicas estables en kilómetros por carretera. No incorpora variables dinámicas en tiempo real, tales como eventos de congestión vehicular, cierres por deslaves o bloqueos carreteros temporales.
2. **Perfil Homogéneo de Navegación:** Las distancias y tiempos fueron computados con el perfil de conducción para vehículo particular ligero (`driving` de OSRM). Rutas de transporte pesado de carga o transporte extraurbano colectivo pueden requerir radios de curvatura mayores o velocidades de crucero considerablemente inferiores.
3. **Simetría y Bidireccionalidad Vial:** El grafo asume aristas bidireccionales de idéntico costo en ambos sentidos ($w(u,v) = w(v,u)$). Si bien esto es rigurosamente cierto en carreteras intermunicipales de dos vías en el altiplano, dentro de cascos urbanos congestionados existen sentidos únicos de circulación que podrían requerir un refinamiento mediante digrafos dirigidos.
4. **Frontera Departamental Rígida:** El grafo restringe sus nodos estrictamente a las 24 cabeceras municipales de Quetzaltenango. En la práctica real, desplazarse entre ciertos municipios periféricos puede resultar más corto transitando por cabeceras de departamentos vecinos (como Totonicapán, San Marcos o Retalhuleu) que no forman parte del conjunto de nodos formal del sistema.

---

10. Conclusiones

1. Se diseñó, implementó y validó exitosamente el sistema computacional **RutaQ**, demostrando la aplicabilidad de los algoritmos clásicos de teoría de grafos (Dijkstra, Floyd-Warshall y Yen) para resolver problemas de optimización de redes viales en el departamento de Quetzaltenango.
2. La implementación del algoritmo de Dijkstra con montículo binario demostró un rendimiento asintótico óptimo ($O((V+E)\log V)$).
3. Se verificó formalmente la exigencia matemática de pesos estrictamente no negativos en el algoritmo de Dijkstra mediante un contraejemplo riguroso, confirmando que la red vial de Quetzaltenango cumple cabalmente con esta precondición al operar con distancias por carretera $km > 0$.
4. El proceso de construcción del dataset geográfico, basado en 276 pares candidatos evaluados exhaustivamente en OSRM y filtrados mediante criterios objetivos de intermediación punto-segmento a 1.0 km, garantizó un grafo 100 % conexo de 24 nodos y 55 aristas, eliminando subjetividades manuales.
5. Las pruebas automatizadas demostraron una concordancia absoluta: (1) coincidencia iteración por iteración al 100 % entre la solución analítica manual y la tabla computada del sistema sobre el grafo de control de 6 nodos, y (2) coincidencia exacta en las distancias del 100 % (576/576 pares) frente a la librería científica NetworkX.

---

11. Manual de Usuario Breve

11.1 Acceso al Sistema y Repositorio Oficial

- **Aplicación Web Desplegada en Producción: https://criss-paz.github.io/rutaq/**
  Disponible en línea sin necesidad de instalación previa. Totalmente responsiva y compatible con navegadores de escritorio, tabletas y teléfonos inteligentes.
- **Repositorio de Código Fuente: https://github.com/Criss-paz/rutaq**
  Contiene la totalidad del código fuente en TypeScript/React, suites de pruebas unitarias y E2E, scripts de generación de datos y documentación técnica.

11.2 Requisitos del Entorno (Ejecución Local)

- Sistema Operativo: Windows 10/11, Linux o macOS.
- Entorno de ejecución: Node.js (versión 18.0 o superior) y gestor de paquetes `npm`.
- Navegador Web: Google Chrome, Mozilla Firefox, Microsoft Edge o Safari con soporte moderno de Canvas y Web Workers.

11.3 Puesta en Marcha Local

Para clonar el repositorio, instalar dependencias y ejecutar la aplicación en un servidor de desarrollo local:

```
# 1. Clonar el repositorio y acceder al directorio
git clone https://github.com/Criss-paz/rutaq.git
cd rutaq

# 2. Instalar dependencias del proyecto
npm install

# 3. Iniciar el servidor de desarrollo Vite
npm run dev
```

Al finalizar la compilación, la terminal mostrará la dirección local (típicamente `http://localhost:5173/`). Abra dicha URL en su navegador web.

11.4 Guía de Operación

1. **Selección de Origen y Destino:**
   - En la barra superior, seleccione el municipio de **Origen** (ej. *Quetzaltenango*) y el municipio de **Destino** (ej. *Coatepeque*), o haga clic directamente sobre los nodos en el mapa.
   - Para consultar el árbol de rutas hacia todos los municipios, elija únicamente un nodo de origen y deje el destino vacío.
2. **Visualización Tabular Paso a Paso (Pestaña "Tabla"):**
   - Haga clic en la pestaña **Tabla** para auditar el funcionamiento de Dijkstra.
   - Use los botones ▶ **Reproducir** o **Paso Siguiente** para avanzar por las iteraciones. Observe cómo se fijan los nodos permanentes (en dorado) y se tachan sus columnas.
3. **Exploración de la Matriz de Floyd-Warshall (Pestaña "Floyd"):**
   - Seleccione la pestaña **Floyd** y desplace el deslizador del vértice intermedio k para observar cómo la matriz $D^{(k)}$ actualiza los costos globales.
4. **Consulta de Rutas Alternativas (Pestaña "Todas las Rutas"):**
   - Active la pestaña **Todas las Rutas** para listar los caminos ordenados por kilometraje. Haga clic sobre cualquier fila para resaltar dicha trayectoria en el mapa interactivo o use el botón **Exportar CSV** para descargar los datos.
5. **Modo Diseño de Grafos (F4):**
   - Presione el botón **Diseñar** en el encabezado para crear nuevos grafos, arrastrar vértices, añadir conexiones personalizadas y probar los algoritmos sobre topologías arbitrarias.

---

12. Referencias Bibliográficas (Normas APA 7)

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
- Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269–271. https://doi.org/10.1007/BF01386390

- Floyd, R. W. (1962). Algorithm 97: Shortest path. *Communications of the ACM*, 5(6), 345. https://doi.org/10.1145/367766.368168
- Gansner, E. R., Koren, Y., & North, S. (2004). Graph drawing by stress majorization. In J. Pach (Ed.), *Graph Drawing: 12th International Symposium, GD 2004* (Lecture Notes in Computer Science, Vol. 3383, pp. 239–250). Springer. https://doi.org/10.1007/978-3-540-31843-9_25
- Instituto Nacional de Estadística [INE]. (2018). *XII Censo Nacional de Población y VII de Vivienda: Resultados del Censo 2018*. República de Guatemala: Instituto Nacional de Estadística.
- Open Source Routing Machine [OSRM]. (2026). *Project OSRM: High-performance routing engine for OpenStreetMap*. https://project-osrm.org/
- OpenStreetMap contributors. (2026). *Planet dump [Data file from OpenStreetMap]*. Open Data Commons Open Database License (ODbL). https://www.openstreetmap.org
- Warshall, S. (1962). A theorem on Boolean matrices. *Journal of the ACM*, 9(1), 11–12. https://doi.org/10.1145/321105.321107
- Yen, J. Y. (1971). Finding the K shortest loopless paths in a network. *Management Science*, 17(11), 712–716. https://doi.org/10.1287/mnsc.17.11.712